

HAAP SDK

Binary Distribution & CrewAI Integration Guide

SDK version 0.1.0-alpha.11 | Protocol: HAAP Canonical Specification v7.2.5 | 2026-05-26

Pre-product alpha. Customer evaluation only.

Contents

1	What Is Distributed	1
2	PyPI Distribution (hawcx-haap)	2
2.1	Install	2
2.2	Build backend: maturin bindings = "bin"	2
2.3	Supported platforms	2
2.4	Locating the binary at runtime	2
2.5	PyPI packages	3
3	npm Distribution (@hawcx/hawcx-haap)	3
3.1	Install	3
3.2	optionalDependencies pattern	3
3.3	Supported platforms	4
3.4	Locating the binary at runtime	4
4	CI Build Matrix	4
5	Getting Started with CrewAI	4
5.1	Prerequisites	4
5.2	Install	5
5.3	Package roles	5
6	Annotated CrewAI Example	5
7	Multi-Tenant Pattern	9
8	Security Model	9

1 What Is Distributed

The HAAP SDK ships **one real binary** — `hawcx-manager` — packaged into platform-specific wheels (PyPI) and scoped npm packages (npm). The binary implements the complete customer-side pipeline described in HAAP CS v7.2.5 §39 (Profile E, five-process pipeline).

Seven legacy names (`haap-supervisor`, `haap-authenticator`, etc.) are preserved as symlinks (Unix) or `.exe` copies (Windows) so existing deployment scripts continue to work unchanged.

Not in this package: `haap-rsv` (MCP-server-side verifier) ships from a separate image `ghcr.io/hawcx/haap-rsv` built from `hx_agent_authorizer`. It is Unix-only by design (UDS + peer-credential checks per §39.12.1).

Table 1: Components dispatched by `hawcx-manager`

Role	Subcommand / Legacy name	Spec reference
Supervisor	<code>supervisor</code> / <code>haap-supervisor</code>	§39.1 — lifecycle orchestrator
Authenticator	<code>authenticator</code> / <code>haap-authenticator</code>	§4.2.1 — holds <code>IK_i</code> , runs X3DH
TQS Precompute	<code>tqs-precompute</code> / <code>haap-tqs-precompute</code>	§40 — session-scoped <code>K_session_root</code>
TQS JIT	<code>tqs-jit</code> / <code>haap-tqs-jit</code>	§40 — request-scoped, 1:1 with Assembler
Assembler	<code>assembler</code> / <code>haap-assembler</code>	§39, §47 — single-flight crypto-proxy
EIB	<code>eib</code> / <code>haap-eib</code>	§45 — External Identity Broker, OAuth bearer
SDK CLI	<code>sdk</code> / <code>haap-sdk</code>	debug / operator CLI

2 PyPI Distribution (`hawcx-haap`)

2.1 Install

```
pip install hawcx-haap
```

On `pip install`, `pip` selects the wheel matching the current platform. `hawcx-manager` lands in `venv/bin/hawcx-manager` (Unix) or `venv\Scripts\hawcx-manager.exe` (Windows).

2.2 Build backend: `maturin bindings = "bin"`

The build backend is `maturin` configured with `bindings = "bin"`. This packages a Rust binary — not a Python extension — into a platform-specific wheel. The same mechanism is used by `ruff`, `uv`, and `black`.

```

1 [build-system]
2 requires = ["maturin>=1.5,<2.0"]
3 build-backend = "maturin"
4
5 [tool.maturin]
6 bindings = "bin"
7 # Rust source lives in sibling hx_agent_client_auth_service repo.
8 # CI passes --manifest-path at build time via release-python.yml.
9 python-source = "src"
```

Listing 1: `python/pyproject.toml` (relevant section)

2.3 Supported platforms

2.4 Locating the binary at runtime

```

1 from hawcx_haap import get_binary_path
2
3 binary = get_binary_path()
```

Table 2: PyPI wheel platform tags

Wheel tag suffix	OS	Architecture
linux_x86_64	Linux	x86-64
manylinux*_aarch64	Linux	ARM64
macosx*_arm64	macOS	ARM64 (Apple Silicon)
win_amd64	Windows	x86-64
win_arm64	Windows	ARM64

```

4 # Unix:      "/path/to/venv/bin/hawcx-manager"
5 # Windows:  "C:\...\venv\Scripts\hawcx-manager.exe"

```

Listing 2: Resolving the bundled binary path

`get_binary_path()` raises `RuntimeError` if the binary is absent (e.g., installed from source without maturin). The error message explains the local development workflow.

2.5 PyPI packages

Package
Role
<code>hawcx-haap</code>
Core IPC client + bundled <code>hawcx-manager</code> binary
<code>hawcx-crewai</code>
CrewAI BaseTool adapter (depends on <code>hawcx-haap</code>)

3 npm Distribution (@hawcx/hawcx-haap)

3.1 Install

```
npm install @hawcx/hawcx-haap
```

npm installs only the platform package matching the current OS and CPU.

3.2 optionalDependencies pattern

The main package declares `optionalDependencies` pointing at five scoped packages, each tagged with `os` and `cpu` fields. npm skips packages for non-matching platforms. The pattern is identical to `@biomejs/biome` and `esbuild`.

```

1 "optionalDependencies": {
2   "@hawcx/hawcx-haap-linux-x64":    "0.1.0-alpha.11",
3   "@hawcx/hawcx-haap-linux-arm64":  "0.1.0-alpha.11",
4   "@hawcx/hawcx-haap-darwin-arm64": "0.1.0-alpha.11",
5   "@hawcx/hawcx-haap-win32-x64":    "0.1.0-alpha.11",
6   "@hawcx/hawcx-haap-win32-arm64":  "0.1.0-alpha.11"
7 }

```

Listing 3: node/package.json (optionalDependencies)

3.3 Supported platforms

Table 3: npm platform packages

npm package	OS	CPU
@hawcx/hawcx-haap-linux-x64	linux	x64
@hawcx/hawcx-haap-linux-arm64	linux	arm64
@hawcx/hawcx-haap-darwin-arm64	darwin	arm64
@hawcx/hawcx-haap-win32-x64	win32	x64
@hawcx/hawcx-haap-win32-arm64	win32	arm64

3.4 Locating the binary at runtime

```

1 import { getBinaryPath } from "@hawcx/hawcx-haap";
2
3 const binary = getBinaryPath();
4 // ->
   "/path/to/.../node_modules/@hawcx/hawcx-haap-linux-x64/hawcx-manager"

```

Listing 4: TypeScript — resolving the bundled binary

4 CI Build Matrix

Both `release-python.yml` and `release-node.yml` run the same five-target Rust build matrix. The binary is checked out from `hx_agent_client_auth_service` and built against the private Kellnr registry at `cargo.hawcx.com`.

Table 4: Rust build targets and CI runners

Rust target	CI runner	PyPI tag	npm pkg
x86_64-unknown-linux-gnu	ubuntu-22.04	linux_x86_64	linux-x64
aarch64-unknown-linux-gnu	ubuntu-22.04-arm	*_aarch64	linux-arm64
aarch64-apple-darwin	macos-14	macosx*_arm64	darwin-arm64
x86_64-pc-windows-msvc	windows-latest	win_amd64	win32-x64
aarch64-pc-windows-msvc	windows-latest	win_arm64	win32-arm64

Releases are triggered by pushing version tags:

```

git tag python-v0.1.0-alpha.11 && git push origin
python-v0.1.0-alpha.11
git tag node-v0.1.0-alpha.11 && git push origin
node-v0.1.0-alpha.11

```

5 Getting Started with CrewAI

5.1 Prerequisites

1. The `hawcx-manager` supervisor pipeline must be running on the agent host. The Assembler's socket follows the convention:

`$XDG_RUNTIME_DIR/hawcx/{agent_id}/agent-assembler-0.sock` (Linux) or
`\\.\pipe\haap-{agent_id}-agent-assembler-0` (Windows).

- The agent identity must be pre-provisioned via the Hawcx Admin Console (CAA → Authenticator flow per CS §4.6.3).

5.2 Install

```
pip install hawcx-haap hawcx-crewai crewai
```

5.3 Package roles

Package	Role
hawcx-haap	IPC client + bundled binary. Core SDK.
hawcx-crewai	HawcxTool — a <code>crewai.tools.BaseTool</code> subclass.
crewai	Multi-agent orchestration framework.

6 Annotated CrewAI Example

The following end-to-end example shows the idiomatic pattern for a HAAP-authenticated CrewAI crew. Annotations explain every security-relevant decision.

Step 1 — Define typed argument schemas

CrewAI surfaces the `args_schema` to the LLM for argument validation and tool-selection prompting. The `user_principal_id` field is the per-call identity axis: the LLM supplies it, and the SDK enforces that it is in the operator-controlled allowlist before writing any IPC bytes (HAAP CS v7.2.5 H-3 hardening).

```

1  from pydantic import BaseModel, Field
2
3  class SearchInput(BaseModel):
4      query: str = Field(
5          description="Search query against the NIM knowledge base."
6      )
7      user_principal_id: str = Field(
8          description=(
9              "End-user on whose behalf the search runs. "
10             "Must be a principal registered for this agent."
11         )
12     )
13
14  class DocsInput(BaseModel):
15      document_id: str = Field(
16          description="Opaque document ID to retrieve."
17     )
18     user_principal_id: str = Field(

```

```

19         description="End-user on whose behalf the document is
20         fetched."
    )

```

Listing 5: Pydantic schemas for tool arguments

Step 2 — Connect to the Assembler

`HawcxAgent.connect_by_agent_id()` resolves the conventional UDS path and performs the §7 capability handshake synchronously. `principal_allowlist` is a required, *operator-controlled* closed set. Never derive it from LLM output or request bodies.

```

1  from hawcx_haap import HawcxAgent
2
3  ALLOWED_USERS = ["alice@example.com", "bob@example.com"]
4
5  with HawcxAgent.connect_by_agent_id(
6      "research-u1",
7      principal_allowlist=ALLOWED_USERS, # required; enforced
8      before any IPC write
9  ) as agent:
10     ...

```

Listing 6: Opening the Assembler IPC connection

Step 3 — Construct `HawcxTool` instances

One `HawcxTool` per logical tool. All instances share the same `agent` (one Assembler connection per process). Construction is cheap — no IPC at this point.

Key fields:

- **provider** — §47.2 provider class. Routes the §47.8 `GetExternalCredential` IPC to the correct sidecar (`haap-nim-provider`, `haap-anthropic-provider`, etc.).
- **tool_id** — §47.4 tool identity binding. The sidecar refuses to disclose a credential unless the requesting `tool_id` matches the bound value. Two tools with different `tool_ids` cannot share credentials even within the same pipeline.
- **endpoint** — Destination URL. The Assembler constructs the outbound HTTPS request; the Python process never sees the bearer token.
- **action** — TBAC action list for Cedar policy evaluation.

```

1  from hawcx_crewai import HawcxTool
2
3  nim_search_tool = HawcxTool(
4      name="nim_search",
5      description=(
6          "Search the organisation's private knowledge base via
7          NVIDIA NIM. "
8          "Always supply the user_principal_id you were given."
9      ),
10     hawcx_agent=agent,
11     provider="nim", # routes to
12     haap-nim-provider sidecar
13     tool_id="nim-search-v1", # credential binding
14     key (sec. 47.4)
15     endpoint="https://api.nim.nvidia.com/v1/search",

```

```

13     method="POST",
14     action=["read"],                                # Cedar policy action
15     args_schema=SearchInput,
16 )
17
18 docs_tool = HawcxTool(
19     name="docs_reader",
20     description=(
21         "Retrieve a document from the internal store by ID. "
22         "Always supply the user_principal_id you were given."
23     ),
24     hawcx_agent=agent,
25     provider="generic-bearer",                      # generic bearer
26     sidecar                                         # separate credential
27     tool_id="docs-reader-v1",                      from nim_search
28     endpoint="https://docs.internal.example.com/v1/documents",
29     method="GET",
30     action=["read"],
31     args_schema=DocsInput,
32 )

```

Listing 7: Constructing HawcxTool instances

Step 4 — Define CrewAI Agents

Each CrewAI Agent receives the relevant HawcxTool(s) in its `tools` list. The LLM backing the agent sees only the tool's name, description, and `args_schema` — never provider keys, HAAP session tokens, or IPC internals.

```

1  from crewai import Agent
2
3  researcher = Agent(
4      role="Research Analyst",
5      goal=(
6          "Find and retrieve the most relevant technical documents "
7          "answering the user's question."
8      ),
9      backstory=(
10         "A meticulous analyst with access to a private knowledge "
11         "base. "
12         "Always cites sources by document ID."
13     ),
14     tools=[nim_search_tool, docs_tool],
15     verbose=True,
16 )
17
18 summarizer = Agent(
19     role="Technical Writer",
20     goal="Synthesize research findings into a concise technical "
21         "summary.",
22     backstory=(
23         "Turns raw research into polished summaries. "
24         "Does not call external tools."
25     ),
26     tools=[],                                # no HAAP tools needed for summarisation
27     verbose=True,
28 )

```

Listing 8: Defining CrewAI agents

Step 5 — Define Tasks

The `user_principal_id` is injected into the task description so the LLM learns to pass it through every tool call. The principal is sourced from the authenticated session — never from LLM output.

```

1 from crewai import Task
2
3 user_id = "alice@example.com"      # from authenticated session
4 question = "What are the performance characteristics of the HAAP
   TQS pipeline?"
5
6 research_task = Task(
7     description=(
8         f"Answer: {question}\n\n"
9         f"IMPORTANT: For every tool call, pass "
10        f"user_principal_id='{user_id}'. Do not omit this field."
11    ),
12    expected_output=(
13        "List of relevant document IDs with key findings and
14        direct quotes."
15    ),
16    agent=researcher,
17 )
18
19 summarize_task = Task(
20     description=(
21         "Using the research findings above, write a 3-paragraph
22         technical "
23         "summary for a developer audience. Include document IDs
24         as citations."
25     ),
26     expected_output="Concise 3-paragraph technical summary with
27     citations.",
28     agent=summarizer,
29     context=[research_task],        # receives researcher's output
30     as context
31 )

```

Listing 9: Task definitions with per-call principal

Step 6 — Assemble and run the Crew

```

1 from crewai import Crew, Process
2
3 crew = Crew(
4     agents=[researcher, summarizer],
5     tasks=[research_task, summarize_task],
6     process=Process.sequential,    # tasks run in order; use
   hierarchical
7     verbose=True,                  # for manager-agent
   architecture
8 )

```



```

9
10 result = crew.kickoff()
11 print(result)
12 # The 'with' block closes the Assembler IPC socket on exit.
13 # Keep the agent open for the lifetime of the process to avoid
14 # the ~5 ms per-reconnect handshake latency.

```

Listing 10: Assembling and running the Crew

7 Multi-Tenant Pattern

When one agent process serves many end-users, two patterns are supported:

A. Per-call principal via *args_schema* (recommended)

Declare `user_principal_id` in the tool's `args_schema`. The LLM supplies it on each call; the SDK enforces it against `principal_allowlist` before any IPC.

```

1 # SearchInput / DocsInput in Step 1 already include
   user_principal_id.
2 # No extra code is needed; the SDK extracts it from kwargs in
   HawcxTool._run.

```

Listing 11: Per-call principal in `args_schema`

B. Per-user tool instance via *for_user*

For flows that materialise one tool per user request:

```

1 # Cheap -- shares the underlying HawcxAgent (one IPC connection).
2 alice_search = nim_search_tool.for_user("alice@example.com")
3 bob_search   = nim_search_tool.for_user("bob@example.com")
4
5 # alice_search always calls invoke_for("alice@example.com", ...)
6 # bob_search   always calls invoke_for("bob@example.com", ...)

```

Listing 12: `for_user` sugar

8 Security Model

Generated 2026-05-26. Source: `hawcx_agentic_sdk 0.1.0-alpha.11`, HAAP Canonical Specification v7.2.5. Copyright © 2026 Hawcx Inc. All rights reserved.

Table 5: HAAP SDK security properties

Property	How it is achieved
Credentials never reach the LLM process	<code>HawcxTool._run</code> calls <code>HawcxAgent.invoke</code> over local UDS only. The Assembler fetches provider credentials internally (§47.8 <code>GetExternalCredential</code>) and attaches them to the outbound HTTPS request. The credential value never returns to Python.
HAAP session keys never reach the LLM process	All crypto (<code>K_session_root</code> , <code>K_req</code> , <code>K_resp</code>) lives inside the Assembler process. The SDK exchanges only plaintext request bodies and decrypted response bodies.
LLM-supplied principals are sandboxed	<code>principal_allowlist</code> is a closed, operator-controlled set validated synchronously at <code>HawcxAgent.connect()</code> time. A compromised or hallucinating LLM cannot escalate to an out-of-list principal.
Per-tool credential binding	<code>tool_id</code> (§47.4) prevents one tool from borrowing another tool's credentials even within the same pipeline.
IPC socket isolation	Sockets are placed under <code>\$XDG_RUNTIME_DIR/hawcx/</code> (0o700, per-UID, systemd-managed). The SDK refuses to fall back to <code>/tmp/hawcx/</code> without an explicit <code>HAAP_SDK_ALLOW_TMP_IPC=1</code> opt-in.